

Raport de Securitate (Mini Pentest) - Proiect 2: Break the Login

Student: Lungu Bogdan-Cosmin

Grupa: 461

1. Introducere

Descriere aplicatie si arhitectura:

Proiectul "AuthX" este o aplicatie web interna folosita de angajati pentru a accesa resurse sensibile (Dashboard). Scopul aplicatiei este de a demonstra in prima instanta un sistem de autentificare functional, dar extrem de vulnerabil (V1), iar ulterior dezvoltarea si securizarea acestuia la standarde moderne (V2).

Arhitectura se bazeaza pe un model Client-Server, unde backend-ul ofera endpoint-uri pentru autentificare, inregistrare, gestionare de sesiuni si resetare parole. Interfata grafica (UI) este simplificata folosind HTML/CSS, intreaga logica de validare si securitate aflandu-se in backend. Conform clarificarilor legate de "Implementare MVP", focusul principal este exclusiv pe functionalitatile de Auth (Authentication), componenta de CRUD/Search pentru tichete fiind ignorata(dupa intrebarea pusa de colegul meu pe canalul de teams), considerand Dashboard-ul ca resursa protejata.

2. Setup Mediu

Instalare VM, DB, framework:

- **Masina Virtuala:** Dezvoltarea si testarea s-au facut pe ultima versiune de Kali Linux (2026.1).
- **User Kali:** bogdan (schimbat ulterior la lungubogdan461)
- **Baza de date:** S-a utilizat PostgreSQL v18, administrat prin interfata online pgAdmin 4 (logat cu contul personal lungubogdan02@gmail.com care se poate vedea si in imagini). Scriptul `db_init.py` a creat tabelele `users` si `password_resets`.
- **Framework si Dependente:** S-a folosit Python 3 izolat intr-un mediu virtual (venv). Librariile instalate sunt: `Flask` (pentru server web), `Flask-Session` (pentru sesiuni pe server-side),

psycopg2-binary (pentru driverul de PostgreSQL) si bcrypt (pentru criptarea moderna a parolelor).

- **Tool-uri de testare:** S-a utilizat Burp Suite (prin browserul integrat in suita) pentru capturarea, analiza si manipularea request-urilor / response-urilor HTTP.
- **IDE:** Visual Studio Code.

Mai jos este scriptul complet de inițializare a bazei de date (`db_init.py`) folosit în acest mediu:

```

import psycopg2
from config import DB_CONFIG

def init_db():
    try:
        # conectare la postgres pt a crea baza de date
        conn_postgres = psycopg2.connect(
            dbname="postgres",
            user=DB_CONFIG["user"],
            password=DB_CONFIG["password"],
            host=DB_CONFIG["host"],
            port=DB_CONFIG["port"]
        )
        conn_postgres.autocommit = True
        cursor = conn_postgres.cursor()

        cursor.execute("SELECT 1 FROM pg_catalog.pg_database WHERE datname = %s", (DB_CONFIG["db"],))
        exists = cursor.fetchone()
        if not exists:
            cursor.execute(f"CREATE DATABASE {DB_CONFIG['dbname']}")
            print("Baza de date a fost creata!")

        cursor.close()
        conn_postgres.close()

        # conectare la baza de date noua creata pt a crea tabelele
        conn = psycopg2.connect(
            dbname=DB_CONFIG["dbname"],
            user=DB_CONFIG["user"],
            password=DB_CONFIG["password"],
            host=DB_CONFIG["host"],
            port=DB_CONFIG["port"]
        )
        cursor = conn.cursor()

        # 3.1 stocare utilizator baza de date reala + asocierea unui rol simplu 'user'
        # 4.3 adaugarea coloanelor failed_logins si locked pt a putea bloca conturi
        cursor.execute("CREATE TABLE IF NOT EXISTS users (id SERIAL PRIMARY KEY, email VARCHAR(255) NOT NULL, password VARCHAR(255) NOT NULL, failed_logins INT DEFAULT 0, locked BOOLEAN DEFAULT FALSE)")

        # tabel pt gestionarea tokenurilor de resetare a parolei (utilizat pt 3.4 / 4.6)
        cursor.execute("CREATE TABLE IF NOT EXISTS password_resets (id SERIAL PRIMARY KEY, user_id INT NOT NULL, token VARCHAR(255) NOT NULL, expires_at TIMESTAMP NOT NULL)")

        conn.commit()

```

```
        cursor.close()
        conn.close()
        print("Tabelele au fost create!")

    except Exception as e:
        print(f"Eroare: {e}")

if __name__ == "__main__":
    init_db()
```

Explicația Schemei Bazei de Date și Securitatea din Design:

Tabelul `users` stochează informațiile esențiale ale utilizatorilor. Observați prezența coloanelor `locked` (Boolean) și `failed_logins` (Integer). Aceste câmpuri au fost proiectate "Secure by Design" încă de la faza de inițializare a bazei de date, permițând implementarea ulterioară a funcționalității de Rate Limiting și Account Lockout direct la nivelul bazei de date. Astfel, starea de blocare persistă chiar și la repornirea serverului, protejând eficient și pe termen lung împotriva atacurilor de tip Brute Force.

Tabelul `password_resets` stochează token-urile pentru funcția de uitare a parolei, având un foreign key (`user_id`) restricționat prin `ON DELETE CASCADE` și un câmp obligatoriu `expires_at` care este vital pentru securitatea funcționalității de resetare.

Rularea aplicațiilor se face rulând scripturile din terminal: `python app_v1.py` (Port 5001) și `python app_v2.py` (Port 5002).

3. Implementare MVP (Auth)

Aplicația implementează flow-ul complet de autentificare (Register, Login, Dashboard protejat, Logout, și Forgot/Reset Password). După cum s-a confirmat, MVP-ul se axează strict pe mecanismele corecte de autentificare, aplicând verificări în baza de date PostgreSQL și returnând erori sau confirmări utilizatorului.

4. Prezentare vulnerabilitati, PoC, Impact, Fix si Re-test

4.1 Password Policy Slab

Prezentare vulnerabilitate:

In versiunea V1, nu exista validare pentru complexitatea parolei, permitand parole triviale de o singura cifra sau litera.

Cod Vulnerabil (extras din app_v1.py):

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']

        # 4.1 vulnerabilitate: password policy slab
        password_hash = hashlib.md5(password.encode()).hexdigest()

        # ... inserare directa in baza de date
```

Demonstrare atac (PoC):

Am inregistrat un cont (email test) cu o parola de doar 3 caractere ("123") in browser.

The screenshot displays a development environment with three main components:

- Left Panel (Project 2 - Break the Login):** Shows a directory structure with files like `_pycache_`, `templates_v1`, `templates_v2`, `app_v1.py`, `app_v2.py`, `auth_db`, `config.py`, and `db_init.py`. Below this is a table titled "Cerințe de securitate" (Security Requirements) with columns for "Categorie" (Category), "Vulnerabilitate inițială (v1)" (Initial Vulnerability), "Atac demonstrat (PoC)" (Demonstrated Attack), and "Fix obligatorie (v2)" (Mandatory Fix).
- Middle Panel (Terminal):** Shows the command prompt for the application, indicating it is running on `http://127.0.0.1:5001`. It also shows the output of the `python app_v1.py` command, which includes a warning about the development server and the start of the application.
- Right Panel (Burp Suite):** Shows the "Register (V1)" endpoint. The "Request" tab displays the raw HTTP request, which includes the email `email@bogdan.ro` and the password `123`. The "Inspector" tab shows the decoded request, confirming the password is `123`.

Analiza impact:

Atacatorul poate rula atacuri de tip Brute Force si Credential Stuffing cu un succes masiv, ghicind parolele slabe instantaneu cu ajutorul listelor.

Implementare fix:

In `app_v2.py` , am adaugat o functie `is_password_complex(password)` apelata pe backend la inregistrare/resetare. Aceasta forteaza o lungime de minim 8 caractere, litere mari/mici, cifre si simboluri speciale.

Cod Securizat (extras din `app_v2.py`):

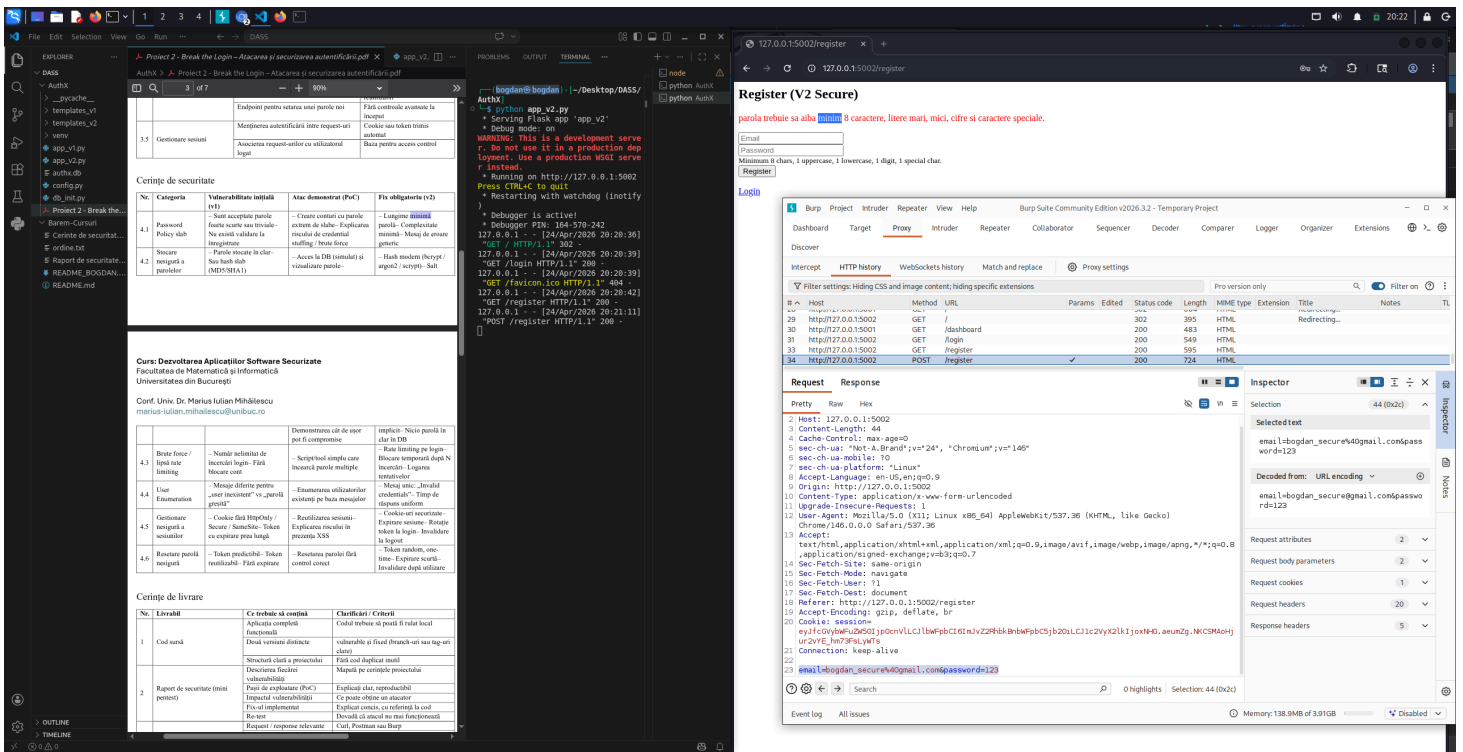
```
def is_password_complex(password):
    # 4.1 lungime minima parola si complexitate minima
    if len(password) < 8: return False
    if not any(char.isdigit() for char in password): return False
    if not any(char.isupper() for char in password): return False
    if not any(char.islower() for char in password): return False
    if not any(char in "!@#$%^&*()-_+=[]{};:,.<>?" for char in password): return False
    return True

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']

        # 4.1 validare la inregistrare - respingere instante slabe
        if not is_password_complex(password):
            return render_template('register.html', error="parola trebuie sa aiba minim 8 caractere")
```

Re-test:

Inercarea de a inregistra o parola simpla pe V2 ("123") este refuzata automat cu mesaj explicativ.



4.2 Stocare nesigura a parolelor

Prezentare vulnerabilitate:

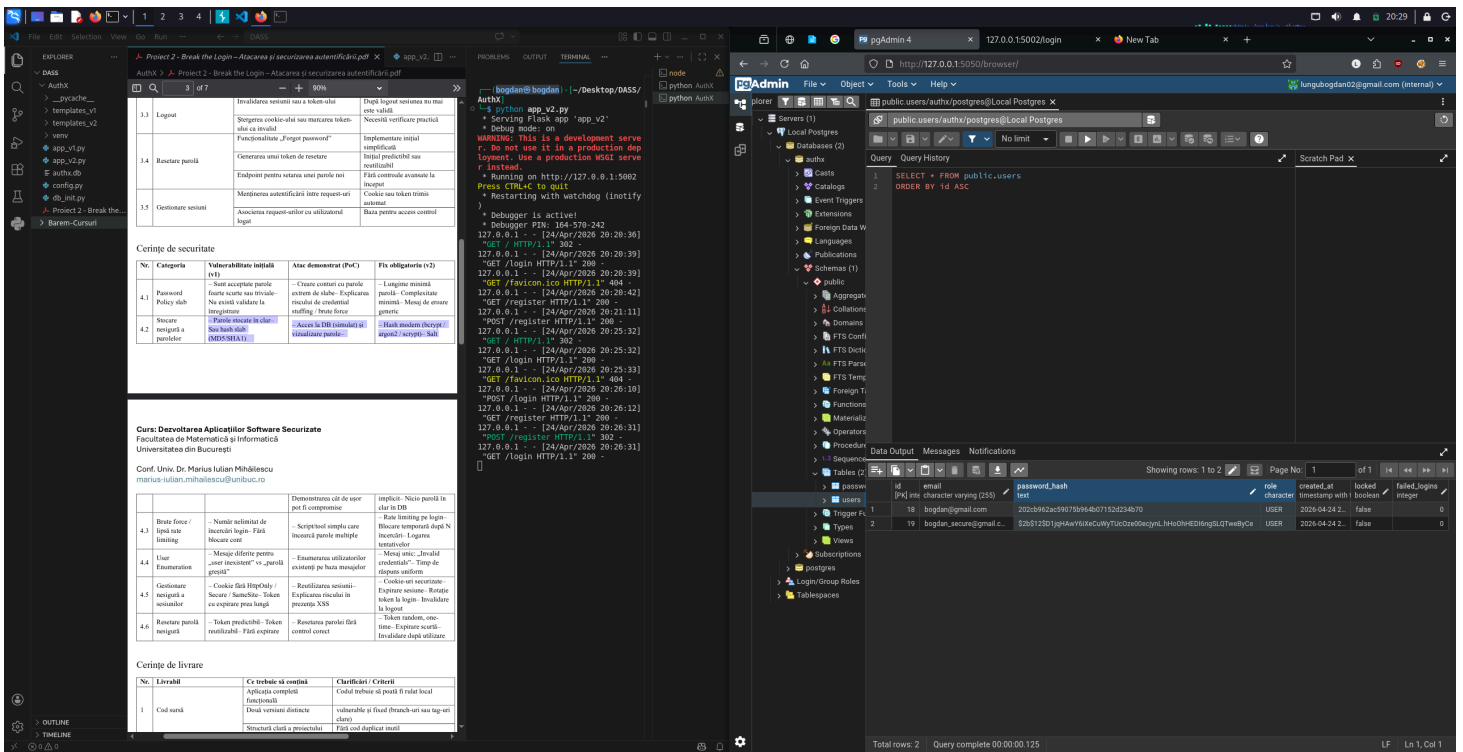
In V1, parolele sunt salvate folosind functia `hashlib.md5()` fara un salt criptografic. MD5 este considerat teoretic si practic "spart" in contextul stocarii de parole.

Cod Vulnerabil (extras din `app_v1.py`):

```
# 4.2 vulnerabilitate: stocare nesigura a parolelor - hash slab (md5) fara salt
password_hash = hashlib.md5(password.encode()).hexdigest()
```

Demonstrare atac (PoC):

Am extras datele din pgAdmin 4. Hash-ul vizibil pentru parola este extrem de scurt, comun tuturor instalatiilor (predictibil).



Analiza impact:

In cazul in care baza de date este expusa (prin SQL Injection sau scurgeri de date), atacatorul poate folosi Rainbow Tables sau hashcat pentru a decripta offline milioane de parole in cateva secunde.

Implementare fix:

In V2, s-a importat biblioteca `bcrypt`. Functia `bcrypt.hashpw` genereaza un "salt" pseudo-randomizat pe 128-bit si implementeaza key stretching (algoritm costisitor ca timp), generand string-uri unice chiar si pentru aceeasi parola. Hashul incepe cu `$2b$`.

Cod Securizat (extras din `app_v2.py`):

```
# 4.2 hash modern (bcrypt) + salt implicit (nici o parola in clar in db)
password_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')
```

Re-test:

Dupa cum se vede in poza cu pgAdmin din V2 (imaginea de mai sus), noul hash este modern, de lungime fixa (60 caractere) si imun la Rainbow Tables.

4.3 Brute force / Lipsa Rate Limiting

Prezentare vulnerabilitate:

Endpoint-ul de `/login` din V1 nu monitorizeaza ritmul cererilor. Se pot trimite zeci de mii de parole

intr-un timp extrem de scurt fara niciun mecanism de blocare.

Cod Vulnerabil (extras din app_v1.py):

```
# 4.3 vulnerabilitate: brute force / lipsa rate limiting - numar nelimitat de incercari
password_hash = hashlib.md5(password.encode()).hexdigest()
if user['password_hash'] != password_hash:
    cur.close()
    conn.close()
    return "parola incorecta"
```

Demonstrare atac (PoC):

Folosind utilitarul Burp Suite -> Intruder (atac Sniper), am incarcat un "payload list" cu 15 parole gresite trimise catre emailul unui utilizator. Toate request-urile au intors "HTTP 200 OK", demonstrand ca atacul curge neintrerupt.

The screenshot displays a multi-paneled development and security environment. On the left, a code editor shows Python code for a login system with a vulnerability in the password hashing logic. Below the code, a document titled 'Curs: Dezvoltarea Aplicațiilor Software Securizate' (Course: Developing Secure Software Applications) is visible, detailing security concepts and attack types. The main right pane shows the Burp Suite Intruder tool in 'Sniper attack' mode. The 'Payloads' tab lists 15 different password payloads. The 'Results' tab shows the attack results, with all 15 requests returning a 200 OK status, indicating a successful brute-force attack. The 'Response' tab shows the raw HTTP response for one of the successful attempts, including headers like 'Content-Type: application/x-www-form-urlencoded' and 'Set-Cookie: session=...'. The bottom status bar indicates the attack is complete with 0 highlights and 4 selections.

Analiza impact:

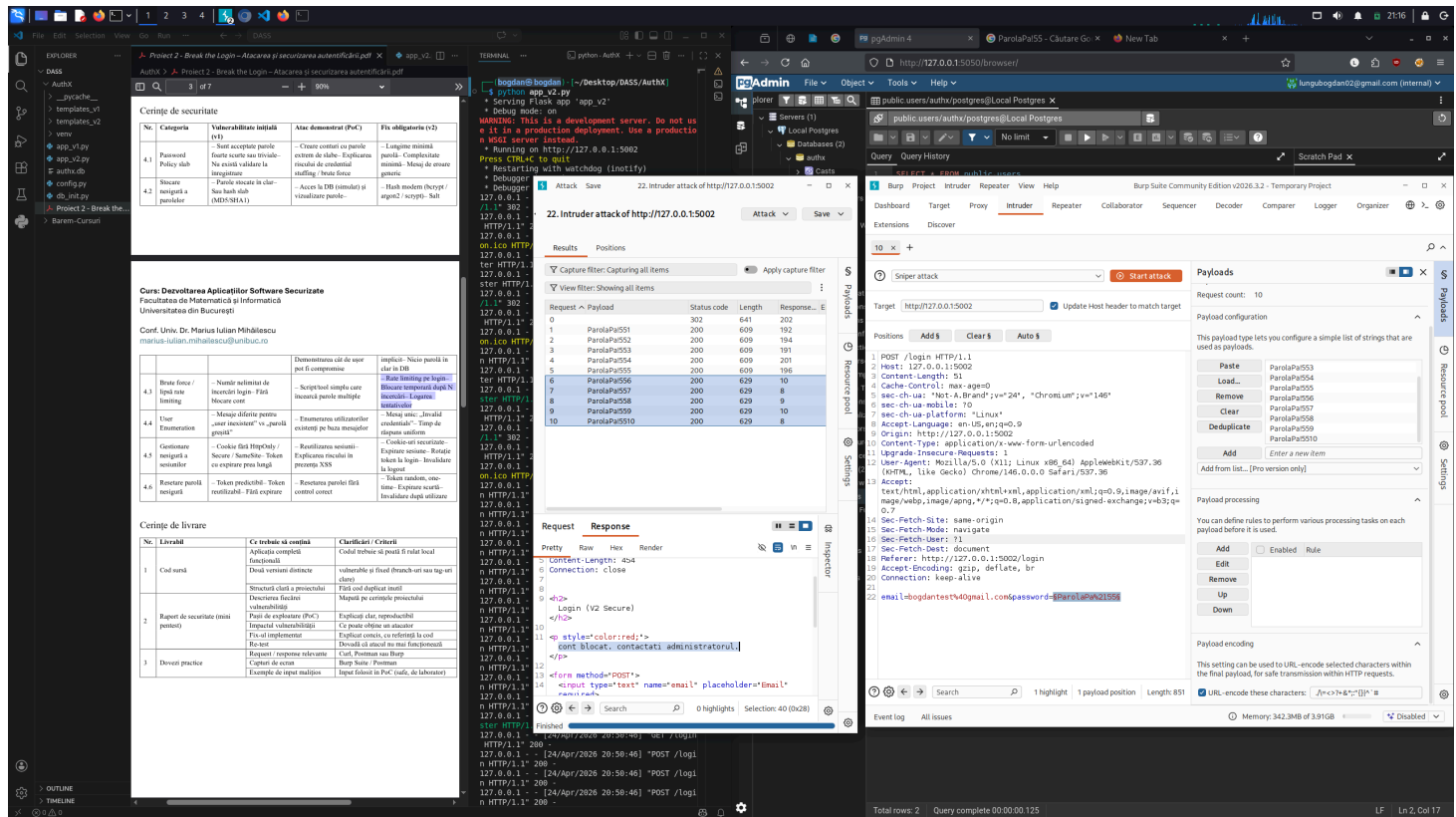
Automatizarea ghicirii parolelor permite compromiterea conturilor care nu respecta politici foarte stricte de complexitate. Atacatorul nu este limitat decat de resursele serverului.

Implementare fix:

Pe V2, rate limiting-ul a fost aplicat direct in baza de date (Account Lockout). Variabila failed_logins se incrementeaza la fiecare esec. Daca numarul de esecuri atinge valoarea 5, coloana locked devine True, iar serverul opreste validările viitoare, returnand "cont blocat".

Re-test:

In Burp Intruder pe V2, se observa ca incepand cu a 5-a incercare, lungimea (Length) raspunsului se schimba brusc, iar corpul raspunsului indica explicit "cont blocat".



4.4 User Enumeration

Prezentare vulnerabilitate:

Mesajele de eroare de la login difera: "utilizator inexistent" cand nu gaseste email-ul si "parola incorecta" cand il gaseste.

Cod Vulnerabil (extras din app_v1.py):

```
# 4.4 vulnerabilitate: user enumeration - mesaje diferite
```

```
if not user:
```

```
    return "utilizator inexistent"
```

```
password_hash = hashlib.md5(password.encode()).hexdigest()
```

```
if user['password_hash'] != password_hash:
```

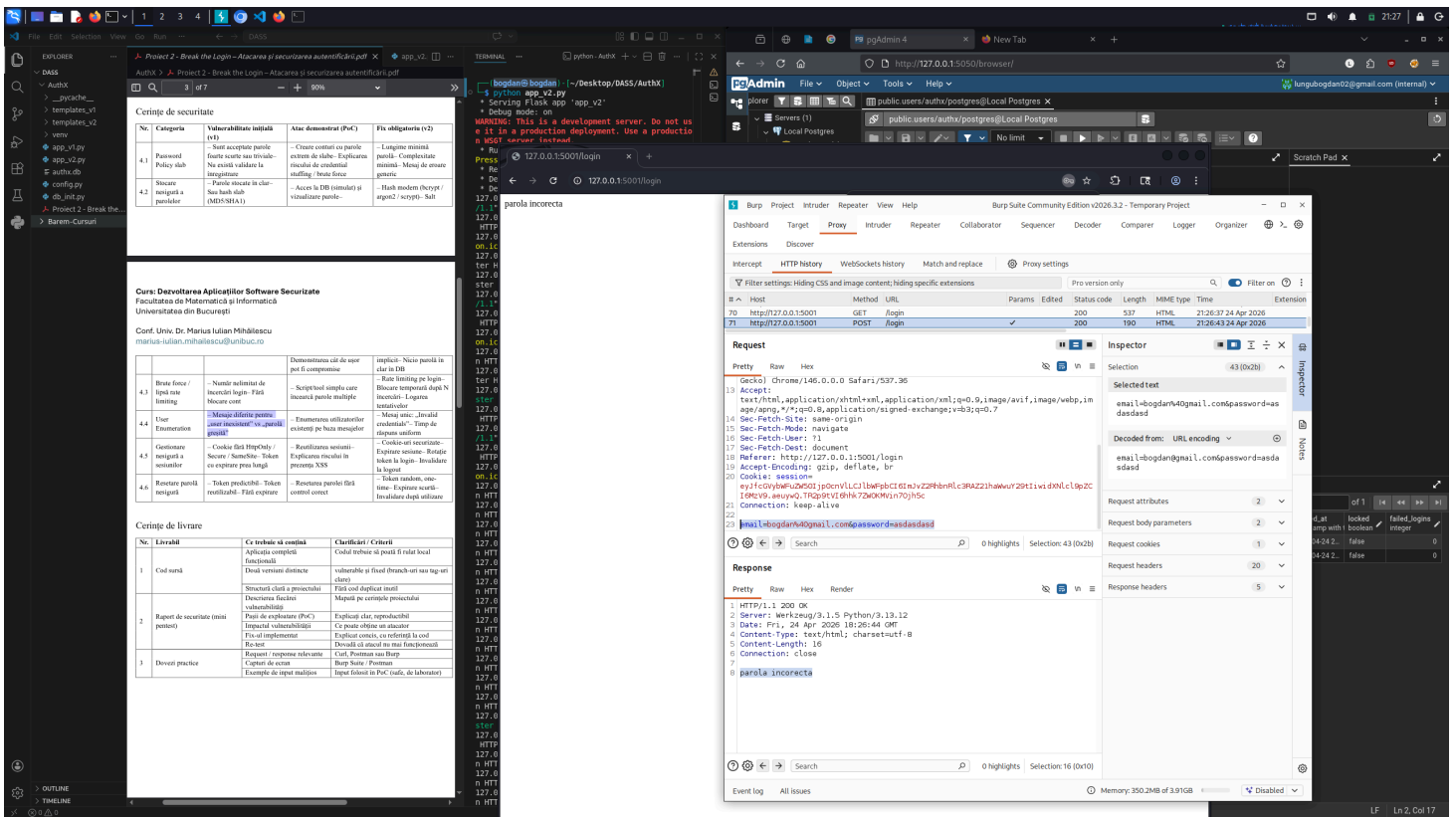
```
    return "parola incorecta"
```

Demonstrare atac (PoC):

Am trimis prin Burp Repeater doua request-uri si am primit raspunsurile diferite descrise mai sus.

The screenshot displays a development environment with three main windows:

- Left Window (PDF):** A document titled "Curs Dezvoltarea Aplicațiilor Software Securizate" (Course: Developing Secure Software Applications) from the Faculty of Mathematics and Informatics, University of Bucharest. It contains sections on security principles and a table of security requirements.
- Middle Window (Code):** A Python script snippet for user enumeration, showing conditional logic for "utilizator inexistent" and "parola incorecta".
- Right Window (Web Browser):** A browser window showing a login page. Below it, the Burp Suite interface is visible, showing a list of HTTP requests and responses. The selected request is a POST to /login with a body containing "utilizator inexistent". The response is a 200 OK status with a body containing "utilizator inexistent".



Analiza impact:

Atacatorul poate incarca liste uriase de adrese de email publice in Burp Intruder pentru a gasi ce utilizatori au conturi in aceasta aplicatie corporativa, construind liste pentru atacuri ulterioare.

Implementare fix:

Am unificat mesajul de raspuns in V2: "credentiale invalide". De asemenea, ca masura avansata de securitate impotriva "Timing Attacks" (enumerare prin timpul de raspuns), daca user-ul nu exista in DB, V2 inca calculeaza un hash dummy folosind `bcrypt.checkpw` inainte de a returna eroarea.

Cod Securizat (extras din `app_v2.py`):

```
# 4.4 anti user enumeration - mesaj unic: "credentiale invalide"
generic_error = "credentiale invalide"
```

```
if not user:
```

```
# 4.4 timp de raspuns uniform - simulam hash-uirea pentru a preveni timing attacks
bcrypt.checkpw(password.encode('utf-8'), bcrypt.gensalt())
cur.close()
conn.close()
return render_template('login.html', error=generic_error)
```

Re-test:

Trimitand aceleasi payload-uri in V2, mesajul ramas pe ecran a fost identic pentru ambele situatii.

Project 2 - Break the Login - Atacarea și securizarea autentificării.pdf

Cură: Dezvoltarea Aplicațiilor Software Securizate
 Facultatea de Matematică și Informatică
 Universitatea din București

Conf. Univ. Dr. Marius Iulian Mihalăscu
 marius.iulian.mihalescu@unibuc.ro

Cerințe de securitate

No.	Categorie	Vulnerabilitate inițială (v1)	Atac demonstrat (PoC)	Ficți obligatorie (v2)
4.1	Password Policy	Sauți acceptate pe parole extrem de slabe - Explicarea: Nu există validare la integrare	- Creare conturi cu parole extrem de slabe - Explicarea: Nu există validare la integrare	- Lungime minimă: 8 caractere - Complexitate minimă: Mixaj de litere, cifre, simboluri, caractere speciale - Hash modern (bcrypt / argon2 / scrypt / salt)
4.2	Autentificare	Acces la DB (token-uri) și validare parole	- Acces la DB (token-uri) și validare parole	

credentiale invalide

Email:

127.0.0.1:5002/login

Register / Forgot Password

Cerințe de livrare

No.	Livrabil	Ce trebuie să conțină	Clarificări / Criterii
1	Analiză	Analiză completă a aplicației	Codul trebuie să poată fi rulat local funcțional
2	Raport de securitate (documentație)	Structură clară a proiectului, vulnerabilități identificate, impactul vulnerabilităților	Mapări pe cerințele proiectului
3	Dezvoltare practică	Implementarea soluțiilor de securizare	Explicarea clar, reproducibilă a soluțiilor de securizare

credentiale invalide

Email:

127.0.0.1:5002/login

Register / Forgot Password

credentiale invalide

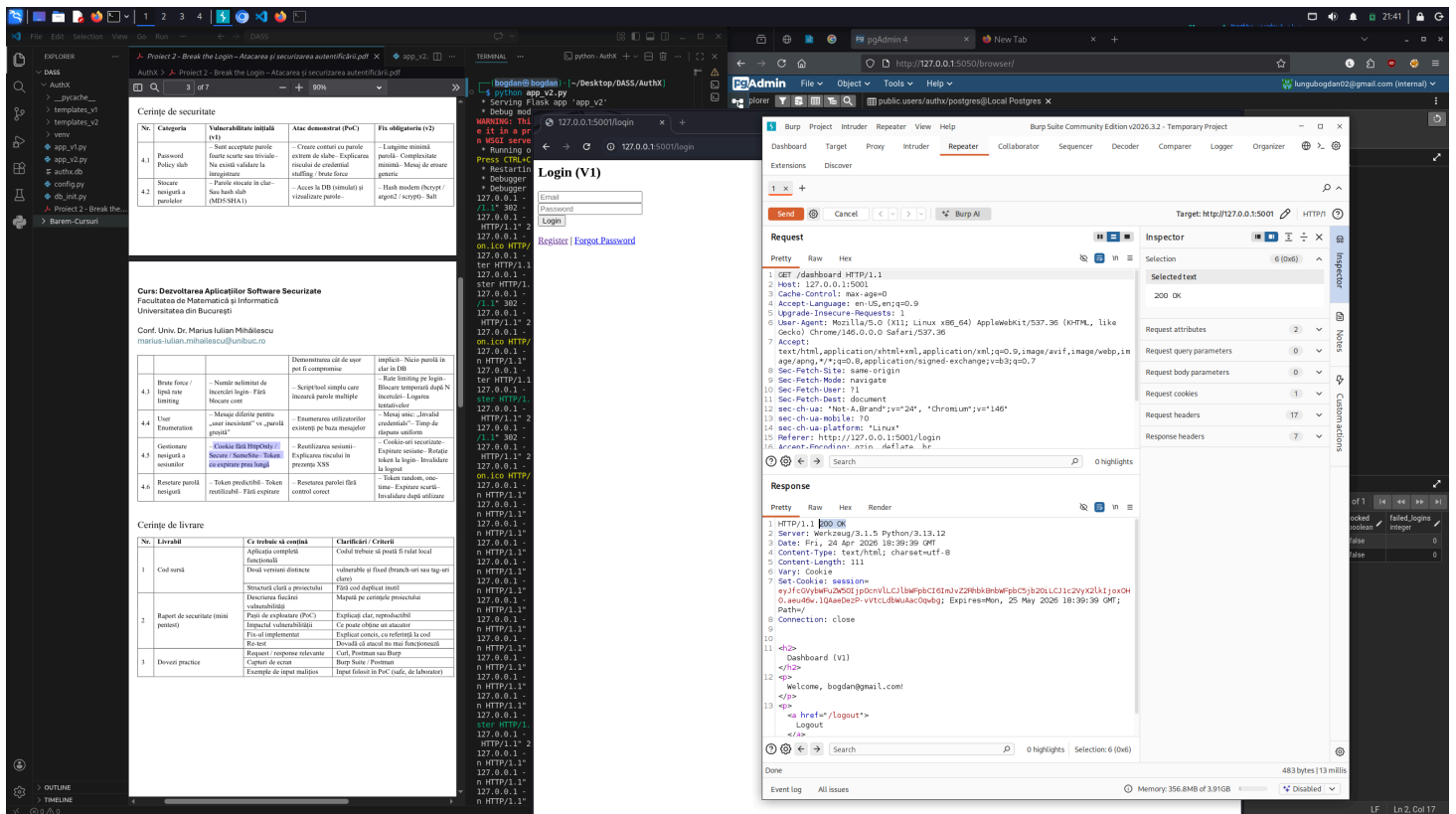
Email:

127.0.0.1:5002/login

Register / Forgot Password

The image shows a development environment with several windows open:

- File Explorer:** Displays the project structure, including folders like 'templates_v2' and files like 'app_v2.py'.
- Table of Security Measures:** A table with columns: 'Categorie', 'Vulnerabilitate initiala (v1)', 'Altece demonstrat (PoC)', and 'Altece demonstrat (v2)'. It lists various security measures like 'Password Policy', 'Session timeout', and 'Hash module'.
- Terminal:** Shows command-line output, including 'python -m http.server' and 'curl' commands.
- Web Browser:** Displays the Burp Suite interface, showing a request and response log. The request is a GET request to '/login' and the response is a 200 status code.



Analiza impact:

Oricine fura token-ul (prin Javascript / sniffing) poate mentine accesul neautorizat mult timp dupa ce victima s-a deconectat, avand in vedere ca timpul de expirare este infinit (permanent).

Implementare fix:

Pentru V2, m-am asigurat ca folosesc extensia Flask-Session pentru "Server-Side Sessions". Acum, cookie-ul de sesiune este generat cu flag-urile HttpOnly , Secure si SameSite=Lax . S-a implementat o expirare scurta (PERMANENT_SESSION_LIFETIME = 30 min). Cel mai important, la delogare (/logout), a fost inclus apelul session.clear() care sterge inregistrarea sesiunii din fisierul de server, anuland utilitatea cookie-ului local.

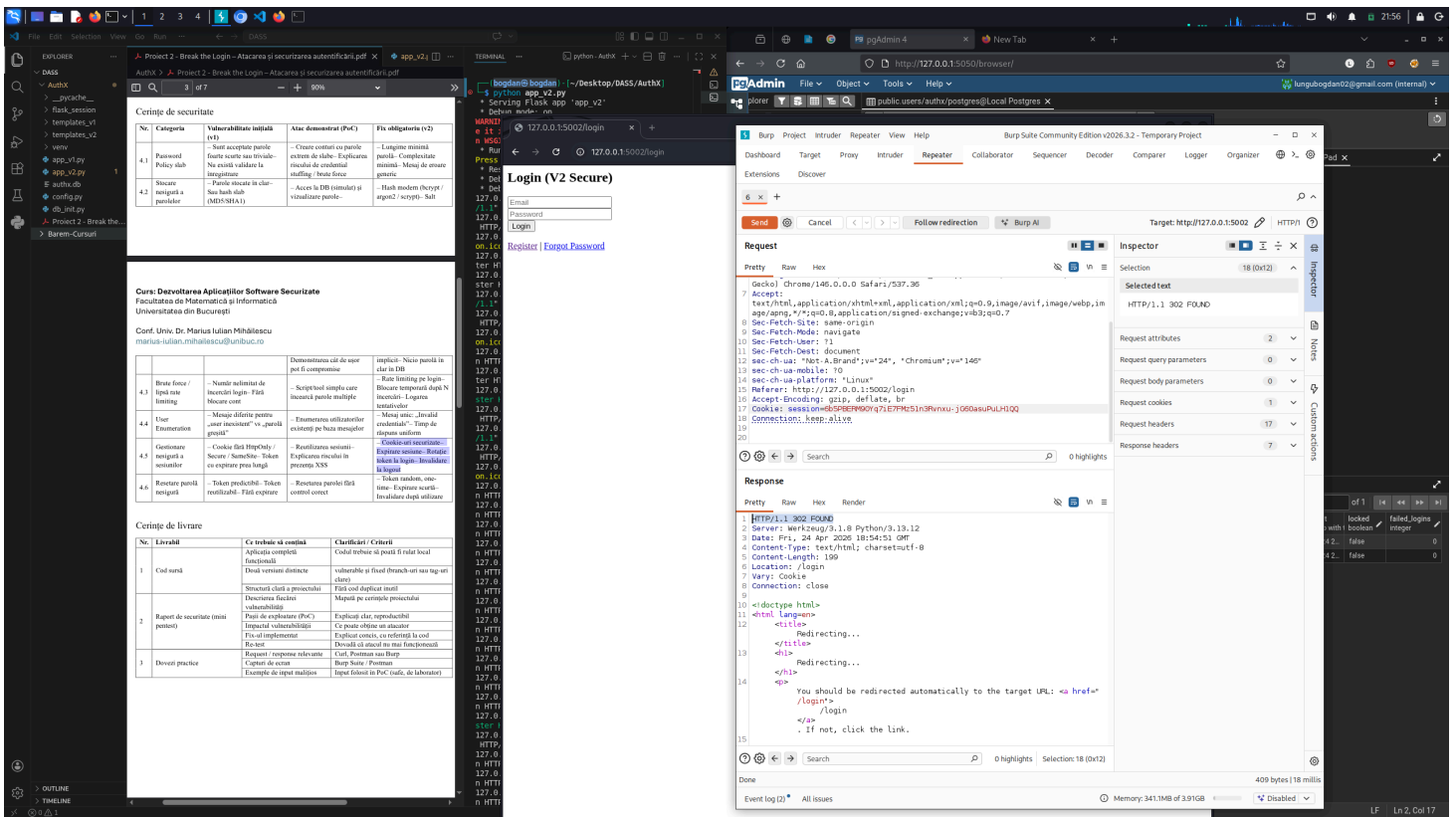
Cod Securizat (extras din app_v2.py):

```
app.config['PERMANENT_SESSION_LIFETIME'] = timedelta(minutes=30)
```

```
return resp
```

Cookie-ul nou returnat are flag-urile necesare. Refolosind vechiul cookie in Burp Repeater dupa logout, serverul V2 refuza complet cererea cu o redirectionare HTTP/1.1 302 FOUND catre pagina de login.

[illegible]



4.6 Resetare parola nesigura

Prezentare vulnerabilitate:

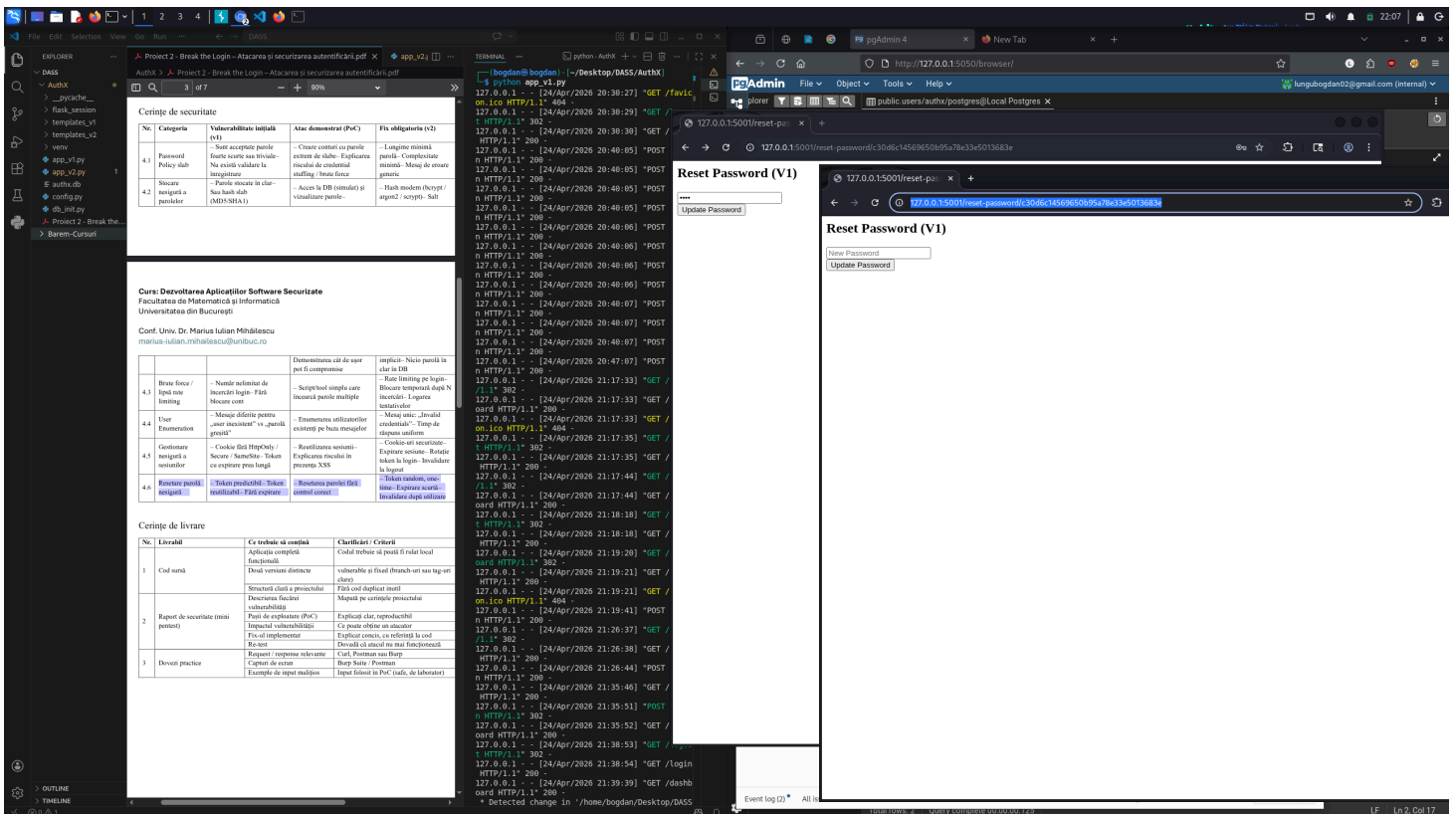
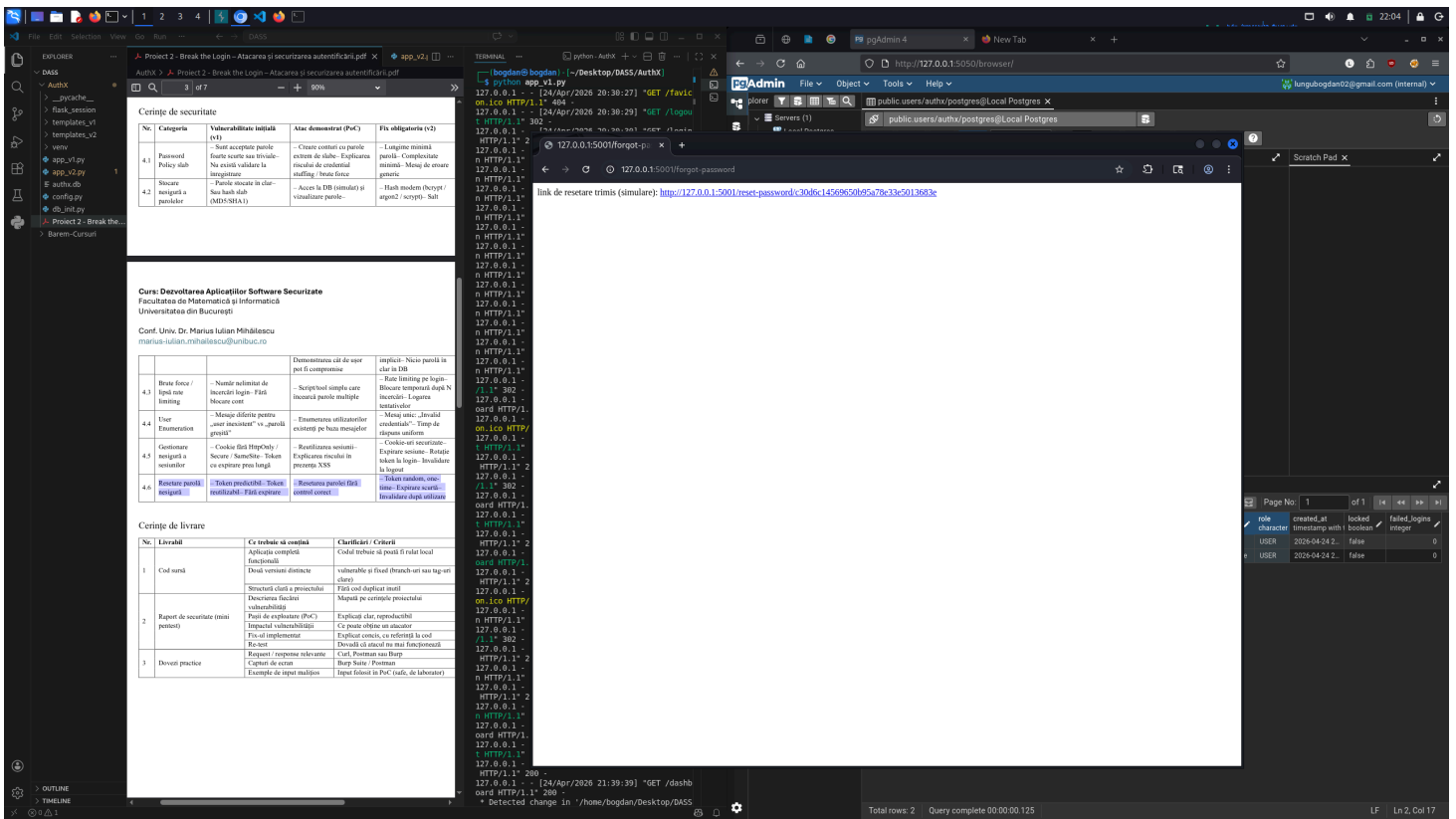
Token-urile de resetare din V1 sunt predictibile si refolosibile la infinit.

Cod Vulnerabil (extras din app_v1.py):

```
# 4.6 vulnerabilitate: resetare parola nesigura - token predictibil (doar un md5 al token = hashlib.md5(email.encode()).hexdigest())  
# 4.6 vulnerabilitate: fara expirare la token (data din viitorul indepartat)  
cur.execute("INSERT INTO password_resets (user_id, token, expires_at) VALUES (%s, %s, %s)"
```

Demonstrare atac (PoC):

Token-ul de resetare din bara de adresa s-a dovedit a fi valoarea de hash MD5 a adresei de email solicitate. Dupa ce am schimbat parola folosind link-ul, i-am dat Refresh paginii. Aplicatia mi-a permis sa rescriu o alta parola, deoarece nu a marcat acel link ca folosit.



Analiza impact:

Predictibilitatea asigura compromiterea prin "Account Takeover" la comanda hacker-ului. Faptul ca nu expira asigura o fereastra de vulnerabilitate permanenta.

Implementare fix:

In V2, token-ul a fost generat criptografic folosind libraria Python `secrets` (`secrets.token_urlsafe`). I-a fost impusa o expirare stricta de 15 minute, salvata in baza de date. Dupa completarea procesului de resetare, am adaugat query-ul de `DELETE` pentru a sterge inregistrarea token-ului (`user_id = %s`), impunand functionalitatea de "One-time use".

Cod Securizat (extras din `app_v2.py`):

```
# 4.6 token random, criptografic sigur
token = secrets.token_urlsafe(32)

# 4.6 expirare scurta (15 minute)
expires_at = datetime.now(timezone.utc) + timedelta(minutes=15)
cur.execute("INSERT INTO password_resets (user_id, token, expires_at) VALUES (%s, %s, %s)"

# In functia reset_password - control one-time use
if request.method == 'POST':
    # ...
    # 4.6 one-time use - invalidare dupa utilizare
    cur.execute("DELETE FROM password_resets WHERE user_id = %s", (reset_entry['user_id'],))
```

Re-test:

Token-ul generat este random. Dupa o resetare reusita, incercarea de a accesa a doua oara acelasi URL s-a soldat cu mesajul "token invalid / expirat." generat din backend.

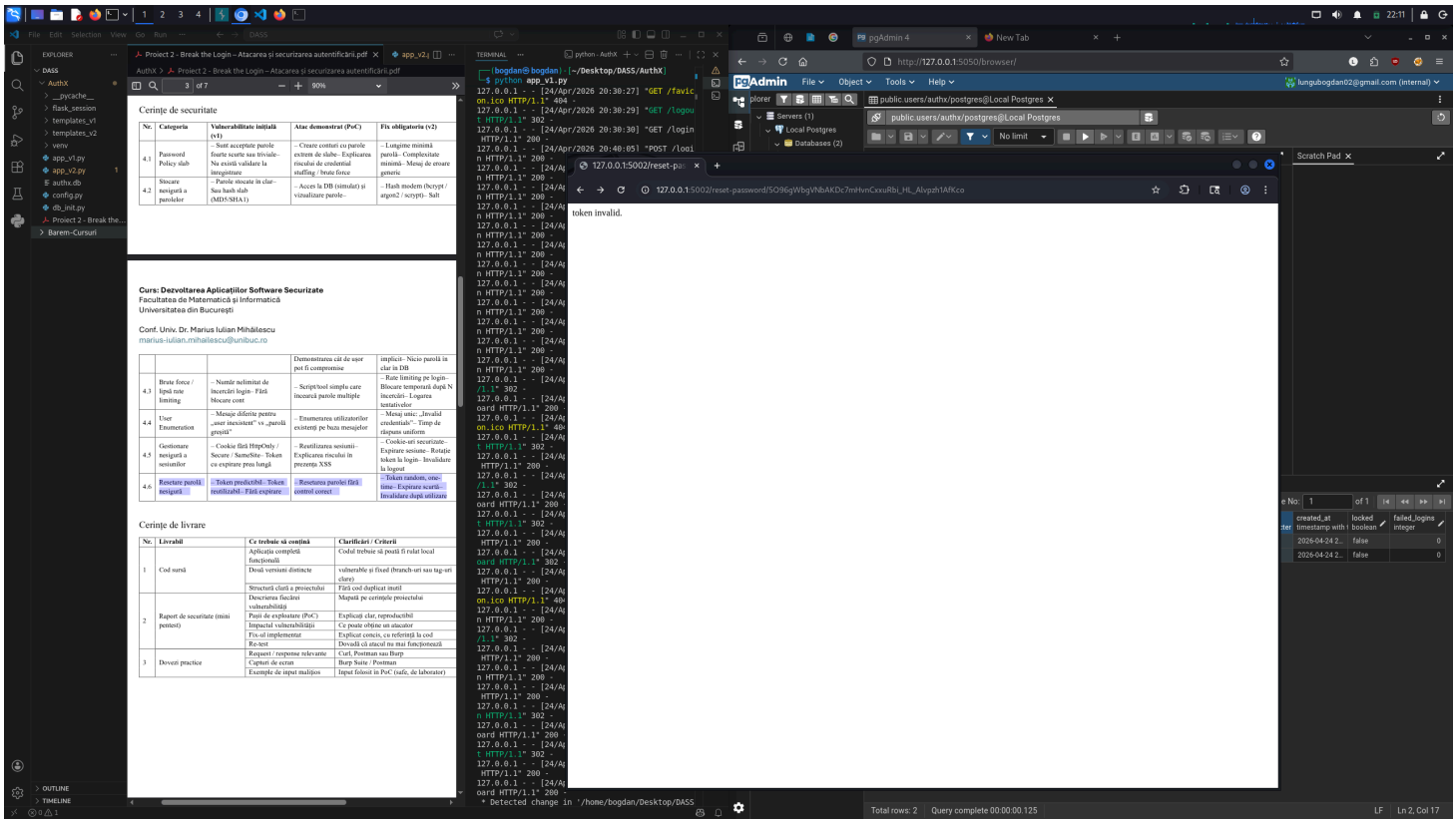
The screenshot displays a development environment with three main components:

- Browser (Left):** Shows a '404 Not Found' error for the URL `http://127.0.0.1:5002/reset-password/`. The error message states: "The requested URL was not found on this server. If you entered the URL manually please check your spelling and try again."
- Terminal (Middle):** Shows the command `curl http://127.0.0.1:5002/reset-password/` being executed, resulting in a 404 status code.
- Document Editor (Right):** Displays a PDF document titled "Certe de securitate" (Security Certificates) from the University of Bucharest. The document outlines security measures and standards, including a table of security requirements and a list of security certificates.

No.	Categorie	Valoriabilitate inițială (v1)	Atac demonstrat (PoC)	Era obligatorie (v2)
4.1	Password Policy slab	- Fără restricții pentru furnizorii de servicii - Nu există validare la înregistrare	- Crearea contului cu parole extrem de slabe - Explicarea riscului de compromitere - Mesaj de eroare generic	- Lungime minimă pentru - Complicație minimă - Mesaj de eroare specific
4.2	Stocare nesigură a parolelor	- Parolele stocate în clar - Securitate slabă (MD5/SHA1)	- Acces la DB (simulată) și vizualizare parole	- Hash modern (bcrypt / argon2 / scrypt - salt)

No.	Descriere	Implementare	Impact
4.3	Breche de securitate / Intrare în sistem	- Niciun mecanism de înregistrare / login - Fără mecanism de înregistrare / login	- Fără limitare pe login - București timp de 15 secunde - Logarea în sistem
4.4	Utilizator	- Mecanism de înregistrare / login - Fără mecanism de înregistrare / login	- Mesaj generic - "Mesaj generic - înregistrare / login - Fără mecanism de înregistrare / login"
4.5	Utilizator	- Mecanism de înregistrare / login - Fără mecanism de înregistrare / login	- Mesaj generic - "Mesaj generic - înregistrare / login - Fără mecanism de înregistrare / login"
4.6	Resursele de securitate	- Token generat random - Fără mecanism de înregistrare / login	- Resursele de securitate generale - Fără mecanism de înregistrare / login

No.	Descriere	Implementare	Impact
1	Cod sursă	- Cod sursă complet - Cod sursă complet	- Cod sursă complet - Cod sursă complet
2	Report de securitate (anul 2020)	- Raport de securitate complet - Fără mecanism de înregistrare / login	- Raport de securitate complet - Fără mecanism de înregistrare / login
3	Dezvoltare practică	- Dezvoltare practică complet - Fără mecanism de înregistrare / login	- Dezvoltare practică complet - Fără mecanism de înregistrare / login



5. Audit & Logging

Conform clarificarilor primite pe canalul de comunicare (Teams) privind ignorarea componentelor de CRUD (tichete), am ales o abordare diferita pentru sistemul de logging. Astfel, in locul tabelului separat `audit_logs` , trasabilitatea evenimentelor de securitate, care este ceruta la sectiunea 'Audit & logging', din structura obligatorie a raportului, este asigurata prin:

- Logging in Baza de Date:** Campul `failed_logins` din tabelul `users` functioneaza ca un mecanism activ de auditare a tentativelor de login. Acesta permite monitorizarea comportamentului malicios si corelarea atacurilor de tip Brute Force direct la nivelul identitatii vizate.
- Logging in Terminal (Flask):** Serverul backend logheaza automat toate cererile HTTP (Request Logs), incluzand adresa IP, timestamp-ul si endpoint-ul accesat. Aceste loguri ofera vizibilitate completa asupra fluxului de date si a posibilelor scanari automate efectuate asupra aplicatiei.

Aceasta structura a fost aleasa pentru a mentine scope-ul proiectului focusat pe mecanismele corecte de Authentication (Auth MVP), eliminand redundanta unui tabel separat de audit intr-un mediu de testare.

6. Concluzii

Acest proiect mi-a oferit perspectiva duala de Offensive si Defensive security asupra unuia dintre cele mai expuse module din web: Autentificarea.

Printre lectiile majore invatate se enumera faptul ca functiile utilitare implicite in framework-uri (precum `session` normal din Flask fara Server-Side sessions si fara flag-uri de `HttpOnly / Secure`) vin cu defecte considerabile out-of-the-box. Securitatea presupune "Defense in Depth" (Aparare in Adancime): blocarea atacului brut pe de-o parte (Rate Limiting), dar si generarea de timpi de raspuns constanti prin pre-calculari de BCrypt false (pentru a elimina scurgerile "tacute" din User Enumeration). Orice functionalitate expusa atacatorilor trebuie restrictionata si validata activ la nivel de server.